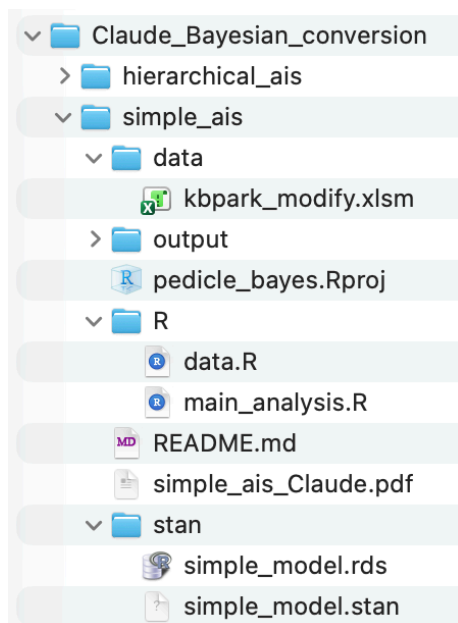


이 논문은 2018년 Spine 지에 게재되었던 PT(Proximal thoracic) curve의 척추경(pedicle)의 넓이에 관한 빈도주의 논문을 베이지 통계로 분석한 논문이다.

앞서 같은 취지로 분석한 논문은 PT curve를 structural curve와 non-structural curve로 구분하지 않은 치명적 약점을 가지고 있다.

이 약점을 보완한 본 논문은 RProject & here 패키지를 사용하며 다음과 같은 구성을 갖는다.



파일을 열려면 Rproject 파일인 pedicle.bates.Rproj을 통해서 열어야 한다. data.R은 레시피 코드로 초반에 한번만 실행하면 되고, main_analysis.R은 main 코드이다. stan 파일은 simple_model.stan이다.

0.1 레시피 코드

레시피 파일은 data.R 파일로 [GitHub 저장소](#)에서 확인할 수 있다. 이 파일은 다음과 같이 나누어서 분석할 수 있다.

1. `cat(here(), "\n\n")`: `here()`는 현재 R 프로젝트의 루트 경로(최상위 폴더)를 반환하는 함수이고, `cat()`는 그 경로를 콘솔에 출력하는 함수이다. `\n`는 줄 띄우기 명령어이다. 따라서 예상 출력은 다음과 같다.

```
/Users/ChoonSungLee/Library/CloudStorage/Dropbox/Claude_Bayesian_conversion/  
simple_ais
```

2. 다음은 data를 불러오는 코드이다. (엑셀 파일 로드)

```
data_path <- here("data", "kbpark_modify.xlsm")
```



```
filter(Level %in% c("T1", "T2", "T3", "T4", "T5", "T6")) %>%
mutate(
  level_idx = as.integer(factor(Level,
                                levels = c("T1", "T2", "T3", "T4", "T5", "T6")))
)
```

이 코드를 분석한다.

- `%>%` 기호는 연결 연산자(pipe operator)로, 이전 결과를 다음 함수의 첫 번째 인수로 전달하는 기능을 한다. R의 그 유명한 tidyverse 패키지에서 제공하는 연산자이다. (내 책 669 페이지, 24.1.3절)
- `clean_df <- raw_df %>% select(ID, 33:66)`: 이 코드는 `raw_df`에서 열 ID, 33:66를 선택하여 `clean_df`라는 데이터프레임을 만들라는 것이다.
- `select(ID, 33:66)`에서 33열은 LtT1, 34열은 LtT2..., 50열은 RtT1,... 66열은 RtL5를 말한다. 따라서 이 코드는 좌우측 중요추의 직경을 나타내는 열을 선택하라는 코드이다.
- `rename_with(~str_remove(., "\\.\.\.\.*"), -ID)`: R(또는 RStudio)에서는 1번 홍추의 직경을 나타내는 LtT1이 (비슷한 이름이 여러 개 있을 때 중복을 피하기 위하여) LtT1...33으로, 2번 홍추의 직경을 나타내는 LtT2는 LtT2...34 등으로 이름이 붙여진다. 따라서 원래의 이름으로 돌려놓기 위하여 ...33 등의 번호를 제거해야 한다.
정규표현식 `\\.\\.\\.\\.\\.*`는 마침표 `.`를 나타내고, `\\.\\.\\.\\.\\.*`는 마침표와 그 이후의 숫자를 나타낸다. 따라서
`rename_with(~str_remove(., "\\.\.\.\.*"), -ID)`는 ID 열을 제외한 나머지 열 이름에서 ...으로 시작하는 뒷부분을 모두 삭제하여 원래 이름(LtT1, LtT2 등)으로 되돌린다. `str`은 string의 약자이다.
- `pivot_longer(cols = -ID, names_to = "key", values_to = "width")`:
`pivot_longer()` 함수는 가로로 나열된 데이터(Wide format)를 세로로 긴 형태(Long format)로 변환시키는데, `cols`, `names_to`, `values_to`의 세 가지 인자가 주로 사용된다.
 - ▷ `cols=-ID`: ID 열은 고정하고,
 - ▷ `names_to = "key"`: LtT1~LtT9 같은 열 이름들은 key라는 새로운 열로 모으고,
 - ▷ `values_to = "width"`: 그 안에 들어있던 실제 측정값(척추경 넓이)은 width라는 열로 재배치하라는 뜻이다.
- `mutate()` 함수는 데이터프레임에서 새로운 열(column)을 만든다.
여기서는 Level, Side_Label의 두 열을 만드는데,
 - ▷ `Level=str_extract(key, "[TL]\\d+")` 코드에서 [TL]은 T 혹은 L로 시작하는 글자를 찾으라는 뜻이고, `\\d+`는 그 뒤에 붙은 숫자(digit)를 모두 가져오라는 뜻이다. 그 결과 LtT1에서는 T1을, RtL5에서는 L5를 쏙 뽑아내어 Level이라는 열에 담는다.

- ▷ `Side_Label=ifelse(str_detect(key, "Lt"), "Left", "Right")`는 key 열의 글자 중에 Lt라는 글자가 포함되어 있는지 찾아본다. 만약 Lt가 발견되면(TRUE) Left라고 쓰고, 발견되지 않으면(FALSE) Right라고 쓰라는 명령어이다.
- `filter()` 함수는 수많은 데이터 중에서 우리가 원하는 조건을 만족하는 진짜 데이터만 남기고 나머지는 걸러내는 거름망 역할을 한다. `filter(A, B)`는 A & B의 두 조건을 동시에 만족해야 한다는 뜻이다.
 - ▷ `filter(!is.na(width), width > 0)` 코드는 width 열의 값이 NA(Not Available) 인지 물어보는 함수로, !(반대 기호)가 붙어서, ‘값이 비어있지 않은 데이터만 통과시켜라’는 뜻이다. 엑셀에서 측정값을 적지 않고 비워둔 행들을 걸러내는 작업이다. 또한 `width>0`에 따라 측정값이 0보다 큰 경우만 남기라는 뜻이다.
 - ▷ `filter(Level %in% c("T1", "T2", "T3", "T4", "T5", "T6"))`: Level 내의 글자가 c()라는 바구니 안에 포함되어 있는 것만 취하라는 뜻이다. `%in%` 연산자는 ‘값 매칭 연산자(value matching operator)’ 또는 ‘멤버십 테스트 연산자(membership test)’로 Level 열의 각 행 값이 지정된 벡터 안의 값들과 일치(match)하는지를 검사하여 (TRUE 또는 FALSE로) 필터링을 수행한다.
- `mutate(level_idx = as.integer(factor(Level, levels = c("T1", "T2", "T3", "T4", "T5", "T6"))))`

이 코드의 의미는 다음과 같다.

- ▷ `factor(Level, levels = c("T1", "T2", "T3", "T4", "T5", "T6"))`는 “이 글자들은 그냥 글자가 아니라 순서와 계급이 있는 집단이다”라고 선언하는 함수이다.
- ▷ `as.integer(...)`는 그 순서를 정수로 바꾸라는 뜻이다.
- ▷ 변환 결과는 다음과 같다.

표 1: 변환 결과 예시

ID	Level (글자)	level_idx (숫자)
P01	T1	1
P01	T2	2
⋮	⋮	⋮
P01	T6	6

여기서 P01은 1번 환자를 나타낸다.

- ▷ 결국 이 코드는 해부학적 용어(T.1~T.6)를 컴퓨터가 이해할 수 있는 숫자 언어(1~6)로 번역한 것이다.
- ▷ `mutate()` 함수는 기본적으로 새로운 컬럼을 추가하는 함수이지만, 컬럼명이 이미 존재하면 덮어쓰고, 없으면 새로 만든다고 이해하면 된다.
- 결국 연결 연산자 `%>%`로 길게 연결된 위의 코드에 의하여 생성된 `clean_df`는 다음과 같다.

```
> clean_df
# A tibble: 2,020 × 6
      ID key   width Level Side_Label level_idx
  <dbl> <chr> <dbl> <chr> <chr>      <int>
1    44 LtT1  4.41 T1    Left         1
2    44 LtT2  3.23 T2    Left         2
3    44 LtT3  2.31 T3    Left         3
4    44 LtT4  2.58 T4    Left         4
5    44 LtT5  2.28 T5    Left         5
6    44 LtT6  2.05 T6    Left         6
7    44 RtT1  4.57 T1    Right        1
8    44 RtT2  3.34 T2    Right        2
9    44 RtT3  3.11 T3    Right        3
10   44 RtT4  3.39 T4    Right        4
# i 2,010 more rows
# i Use `print(n = ...)` to see more rows
```

- 여기서 clean_df의 차원은 2020*6인데 왜 행의 숫자가 2020이 나오는지 궁금해진다. 이 숫자는 다음 세 가지 요소의 조합으로 결정된다.

- ▷ 환자 수: 약 182명
- ▷ 측정 부위 (Level): T1부터 T6까지 총 6개 부위
- ▷ 측정 방향 (Side): 좌, 우 2개 방향

이 요소들을 곱하면 데이터 총합은 다음과 같다. $182 \times 6 \times 2 = 2184$ 개.

여기서 측정값이 비어있거나 유효하지 않은 데이터가 filter 명령에 따라 !is.na(width), width(>0) 코드가 실행되면서 필터링되어 최종적으로 2020개의 행만 남게 된 것이다.

4. 다음은 sPT/non-sPT 그룹 정보 합치기이다.

2018년 논문에서 PT curve의 각도가 side-bending radiographs에서 25° 미만으로 교정되지 않거나, T2~T5 segmental kyphosis > 20°이면 structural curve로 정의했다.

182명의 환자 중 69명이 structural curve(=sPT)였고, 113명이 non-structural curve(=non-sPT)였다.

엑셀 파일 “kbpark_modify.xlsm”의 첫 번째 열인 A열의 컬럼 명이 Gr이며, 이 컬럼에서 숫자 1=non-sPT, 2=sPT로 분류하였다.

```
clean_df <- clean_df %>%
  select(-starts_with("is_structural"))
```

```
group_df <- raw_df %>%
  select(ID, Gr) %>%
  rename(is_structural = Gr) %>%
  mutate(is_structural = case_when(
```

```

is_structural == 2 ~ 1L,    # 2 → sPT (structural)
is_structural == 1 ~ 0L,    # 1 → non-sPT
TRUE                      ~ NA_integer_
))

```

이 코드를 분석해보자.

- 맨 앞의 `select(-starts_with("is_structural"))` 코드는 data.R 코드가 여러 번 실행되면서 `is_structural` 컬럼이 여러 개 생기는 것을 예방하는 코드이다.
여러 개 생기면 콘솔에서 `rm(list = ls())`을 하여 메모리의 모든 변수를 삭제한 후 `source(here("R", "data.R"))` 코드로 data.R을 딱 한 번만 실행하면 된다.
- 원본 엑셀 파일인 `raw_df`에서 첫 번째, 두 번째 열인 Gr, ID를 선택한 후, 컬럼명 Gr을 `is_structural`로 바꾸고, sPT인 `is_structural==2`를 1로 바꾸고, non-sPT인 `is_structural==1`을 0으로 바꾸고, 그 이외의 경우(=TRUE)는 NA로 처리하여 `group_df`라는 데이터프레임을 만들라는 코드이다.
- 따라서 `is_structural==1`이 69명, `is_structural==0`이 113명이 된다.
- Bayesian 분석에서 prior 설정할 때와 brms 패키지 등에서는 이진 변수를 0/1로 가정하고 내부 계산을 수행하기 때문에 이와 같은 변환을 하는 것이다.

```

clean_df <- clean_df %>%
  left_join(group_df, by = "ID")

```

```
cat("그룹 정보 합치기 완료\n")
```

이 코드를 분석해보자.

- 두 개의 데이터프레임(`clean_df`와 `group_df`)을 ID를 기준으로 합치는 작업이다.
- `left_join`의 의미는 왼쪽 데이터인 `clean_df`를 기준으로 (같은 ID를 가진 행끼리) `group_df`를 합친다는 뜻이다.
- `left_join`한 `clean_df`는 다음과 같이 나타난다.

```

> clean_df
# A tibble: 2,020 × 7
   ID key   width Level Side_Label level_idx is_structural
  <dbl> <chr> <dbl> <chr> <chr>      <int>      <int>
1    44 LtT1  4.41 T1    Left         1          0
2    44 LtT2  3.23 T2    Left         2          0
3    44 LtT3  2.31 T3    Left         3          0
4    44 LtT4  2.58 T4    Left         4          0
5    44 LtT5  2.28 T5    Left         5          0
6    44 LtT6  2.05 T6    Left         6          0
7    44 RtT1  4.57 T1    Right        1          0
8    44 RtT2  3.34 T2    Right        2          0
9    44 RtT3  3.11 T3    Right        3          0
10   44 RtT4  3.39 T4    Right        4          0
# i 2,010 more rows

```

```
# 그룹 분포 확인
cat("\n=== 환자별 그룹 분포 ===\n")
clean_df %>%
  distinct(ID, is_structural) %>%
  count(is_structural) %>%
  mutate(Group = ifelse(is_structural == 1, "sPT", "non-sPT")) %>%
  print()
```

이 코드는 합치기가 제대로 됐는지 sPT/non-sPT 환자 수를 확인하는 검증 코드이다.

- `distinct(ID, is_structural)`: 각 환자(ID)당 하나의 행만 남기는 코드다. 각 환자는 T1~L5까지 여러 추체의 데이터가 있으므로 같은 ID가 여러 행에 반복된다. 환자 수를 세려면 중복을 먼저 제거해야 한다.
- `count(is_structural)`: 0과 1 각각 몇 명인지 집계한다.
- `mutate(Group = ifelse(is_structural == 1, "sPT", "non-sPT"))`: `Group`이라는 컬럼을 만들어 `is_structural`이 1이면 sPT, 아니면 non-sPT로 한다는 뜻이다. 다음과 같은 결과가 나온다.

```
# A tibble: 2 × 3
  is_structural     n Group
      <int> <int> <chr>
1         0   113 non-sPT
2         1    69  sPT
```

5. Stan 데이터 리스트 생성: 조금 후 main 파일에서 `stan()` 함수로 fitting을 하는데 이때 `stan_data`가 필요하다. `stan_data`를 지금 생성한다.

```
stan_data <- list(
  N = nrow(clean_df),
  width = as.numeric(clean_df$width),
  vertebra_level_numeric = clean_df$level_idx,
  side_numeric = ifelse(clean_df$Side_Label == "Left", 1, 2)
  structural = as.integer(clean_df$is_structural)
)
```

```
cat("\n=== stan_data 구조 확인 ===\n")
cat("총 관측치(N)      :", stan_data$N, "\n")
cat("sPT 관측치        :", sum(stan_data$structural == 1), "\n")
cat("non-sPT 관측치     :", sum(stan_data$structural == 0), "\n")
```

```
cat("\n[완료] stan_data 준비 완료.\n")
cat("다음 단계: main_analysis.R 실행\n")
```

결과는 다음과 같다.

```
> cat("\n=== stan_data 구조 확인 ===\n")

=== stan_data 구조 확인 ===
> cat("총 관측치(N)      :", stan_data$N, "\n")
총 관측치(N)      : 2020

> cat("sPT 관측치      :", sum(stan_data$structural == 1), "\n")
sPT 관측치      : 739

> cat("non-sPT 관측치   :", sum(stan_data$structural == 0), "\n")
non-sPT 관측치   : 1281

> cat("\n[완료] stan_data 준비 완료.\n")
[완료] stan_data 준비 완료.

> cat("다음 단계: main_analysis.R 실행\n")
다음 단계: main_analysis.R 실행
```

- side_numeric = ifelse(clean_df\$Side_Label == "Left", 1, 2)에서 left는 1로, right는 2로 한다.

0.2 스탠 코드

Stan 파일명은 simple_model.stan으로 [GitHub 저장소](#)의 simple_ais 폴더에 있다. 코드는 다음과 같다.

```
// This model estimates a mean pedicle width difference
//                               for each vertebral level (T1-T6).

data {
  int<lower=0> N;
  array[N] int<lower=1, upper=6> vertebra_level_numeric;
  array[N] int<lower=1, upper=2> side_numeric; // 1: Left, 2: Right
  array[N] real width; // 척추경의 절대 넓이 (mm)
```



```

    array[N] int<lower=0, upper=1> structural;    // 1=sPT, 0=non-sPT
}

parameters {
    matrix[6, 2] mu_sPT;        // sPT 그룹: 6레벨 × 2측면
    matrix[6, 2] mu_nonsPT;    // non-sPT 그룹: 6레벨 × 2측면
    real<lower=0> sigma;
}

model {
    // 사전분포
    to_vector(mu_sPT) ~ normal(5, 3);
    to_vector(mu_nonsPT) ~ normal(5, 3);
    sigma ~ exponential(1);

    // 가능도
    for (i in 1:N) {
        real mu_group = (structural[i] == 1)
            ? mu_sPT[vertebra_level_numeric[i], side_numeric[i]]
            : mu_nonsPT[vertebra_level_numeric[i], side_numeric[i]];
        width[i] ~ normal(mu_group, sigma);
    }
}

generated quantities {

    // 1. 사후예측 (모델 검증용)
    array[N] real y_rep;
    for (n in 1:N) {
        real mu_group = (structural[n] == 1)
            ? mu_sPT[vertebra_level_numeric[n], side_numeric[n]]
            : mu_nonsPT[vertebra_level_numeric[n], side_numeric[n]];
        y_rep[n] = normal_rng(mu_group, sigma);
    }

    // 2. Bayesian p값 (모델 적합성 평가)
    real mean_y_rep = mean(y_rep);
    real mean_width = mean(to_vector(width));
    int p_val      = (mean_y_rep > mean_width);
}

```

```

// 3. 그룹별 2mm 미만 확률
matrix[6, 2] prob_narrow_sPT;
matrix[6, 2] prob_narrow_nonsPT;
for (k in 1:6) {
  for (s in 1:2) {
    prob_narrow_sPT[k, s] = (mu_sPT[k, s] < 2.0);
    prob_narrow_nonsPT[k, s] = (mu_nonsPT[k, s] < 2.0);
  }
}

// 4. 그룹 간 차이 (sPT - non-sPT)
// 음수 = sPT가 더 좁음 (논문의 핵심 발견)
matrix[6, 2] mu_diff;
for (k in 1:6) {
  for (s in 1:2) {
    mu_diff[k, s] = mu_sPT[k, s] - mu_nonsPT[k, s];
  }
}

// 5. LOO-CV용 log_lik
vector[N] log_lik;
for (n in 1:N) {
  real mu_group = (structural[n] == 1)
    ? mu_sPT[vertebra_level_numeric[n], side_numeric[n]]
    : mu_nonsPT[vertebra_level_numeric[n], side_numeric[n]];
  log_lik[n] = normal_lpdf(width[n] | mu_group, sigma);
}
}

```

stan 파일의 코드를 살펴 보자.

- data 블록에서는 분석에 필요한 데이터의 구조를 선언한다. 여기서는 `vertebra_level_numeric`과 `side_numeric`, `width`, `structural`의 네 가지 변수가 정의되어, 각 데이터 포인트의 해부학적 위치 정보, 측정값, 그리고 그룹 구분 정보를 담게 된다..
- parameters 블록에서 정의한 모수(parameter)는 `mu_sPT`, `mu_nonsPT`, `sigma`의 셋이다. `mu_sPT`는 구조성 PT curve의 척추경의 폭(`width`)을 나타내며, `mu_nonsPT`는 비구조성 PT curve의 척추경의 폭(`width`)을 나타낸다. 두 모수 각각 6개 레벨 $\times$ 2개 측면으로 각 12개씩, 총 24개의 평균값을 추정한다. `sigma`는 두 그룹이 공유하는 표준편차이다.

- model 블록의 `to_vector(mu_sPT)`, `to_vector(mu_nonsPT)`, `sigma`는 사전분포(prior distribution)를 나타낸다.
 - ▷ 사전분포는 데이터를 관찰하기 전, 우리가 모델의 모수(parameter)에 대해 가지고 있는 배경 지식이나 가정을 나타낸다.
 - ▷ `mu`는 행렬 형태를 취하는데, Stan은 행렬 자체를 정규분포에 할당하는 것을 허용하지 않는다.
`to_vector(mu)` 함수는 행렬(matrix) 형태인 `mu`를 하나의 긴 열벡터로 변환하여, 모든 원소에 사전분포를 선언하기 위하여 사용한다.
 - ▷ `to_vector(mu\_sPT) ~ normal(5, 3)`: 구조성 PT curve의 척추경의 폭 `mu_sPT`가 정규분포 $\mathcal{N}(5, 3)$ 을 따를 것이라고 사전에 가정하는 것이다. 척추경의 폭이 대략 5mm 내외일 것이라는 의학적 경험을 반영하는 것으로, 베이지 통계에서 말하는 ‘사전분포에 전문가 지식(Expert knowledge)을 반영하는 것’을 잘 보여주는 예시가 된다.
 - ▷ `sigma ~ exponential(1)`: 오차의 표준편차인 `sigma`가 지수분포를 따를 것이라고 가정하는 것이다. `sigma`는 항상 양수이므로 지수분포를 주로 사용한다.
- 모델 블록의 가능도 부분에 대해서 알아본다.

```
for (i in 1:N) {
  real mu_group = (structural[i] == 1)
    ? mu_sPT[vertebra_level_numeric[i], side_numeric[i]]
    : mu_nonsPT[vertebra_level_numeric[i], side_numeric[i]];
  width[i] ~ normal(mu_group, sigma);
}
```

- ▷ 가능도는 모수가 주어졌을 때 관측된 데이터(width)가 나타날 확률을 의미한다.
- ▷ 가능도는 다음과 같이 if-else 문을 많이 사용한다.

```
// if-else 버전
real mu_group;
if (structural[i] == 1) {
  mu_group = mu_sPT[vertebra_level_numeric[i], side_numeric[i]];
} else {
  mu_group = mu_nonsPT[vertebra_level_numeric[i], side_numeric[i]];
}
width[i] ~ normal(mu_group, sigma);
}
```

- ▷ 여기서는 가능도를 삼항 연산자를 사용해서 표현하였다. 완전히 같은 의미이다.

```
// 삼항 연산자 버전 (if-else와 완전히 같은 의미)
real mu_group = (structural[i] == 1)
```

```

? mu_sPT[vertebra_level_numeric[i], side_numeric[i]]
: mu_nonsPT[vertebra_level_numeric[i], side_numeric[i]];
width[i] ~ normal(mu_group, sigma);

```

삼항 연산자는 항이 3개로, “(1) 조건, (2) ? 값 A, (3) : 값 B”의 세 개의 피연산자로 구성되어 있다. ? 는 맞으면, : 는 아니라면(else)를 의미한다. if-else와 완전히 동일한 표현을 한 줄로 압축한 것이다.

- ▷ width[i]는 우리가 실제로 측정한 데이터(척추경의 폭)이다. 이 데이터가 $\mu_{jk}^{(g)}$ 라는 평균과 σ 라는 표준편차를 가진 정규분포에서 추출되었다고 가정하고 있다. 여기서 g 는 해당 관측치의 그룹(구조성/비구조성)을 나타낸다. 관측치 하나하나를 확률밀도함수로 나타낼 수 있으며, *i.i.d.* 가정 하에서 가능도는 각 데이터 포인트의 pdf의 곱으로 쓸 수 있다.

(참고) *i.i.d.*=‘independent and identically distributed’의 약자로, 모든 관측치가 독립적이며 동일한 확률 분포를 가진다는 가정이다. (내 책 48페이지를 볼 것)

- ▷ 개별 관측치 i 에 대한 확률 밀도 함수는 (정규분포를 따르므로) 다음과 같다.

$$f(\text{width}_i | \mu_{jk}^{(g)}, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\text{width}_i - \mu_{jk}^{(g)})^2}{2\sigma^2}\right)$$

여기서

- * j = vertebra_level_numeric[i] (해당 추체의 마디 인덱스)
- * k = side_numeric[i] (해당 추체의 좌우 인덱스)
- * g 는 그룹 구분으로, structural[i] = 1이면 $\mu_{jk}^{(g)} = \mu_{jk}^{\text{sPT}}$, 그렇지 않으면 $\mu_{jk}^{(g)} = \mu_{jk}^{\text{nonsPT}}$ 이다.
- * $\mu_{jk}^{(g)}$ 는 해당 그룹에서 해당 추체의 척추경 폭의 평균값이다.

- ▷ 개별 관측치의 확률 밀도 함수는 일반적으로 다음과 같이 로그 확률 밀도(log-PDF)로 변환하여 사용한다.

$$\log f(\text{width}_i | \mu_{jk}^{(g)}, \sigma) = -\frac{1}{2} \left[\log(2\pi\sigma^2) + \frac{(\text{width}_i - \mu_{jk}^{(g)})^2}{\sigma^2} \right]$$

여기서 좌변은 모수 $\mu_{jk}^{(g)}$ 와 σ 가 주어졌을 때 관측치 width_i 가 나타날 로그 확률 밀도를 의미하며, 이는 Stan 코드에서 normal_lpdf 함수가 계산하는 실제 값과 일치한다.

- ▷ Stan에서 가능도(Likelihood)를 계산하는 방식은 크게 세 가지로 표현할 수 있으며, 이들 모두 내부적으로 로그 확률 밀도(log-PDF)를 합산하는 동일한 원리를 가진다.

- * **Sampling Statement (~) 방식:** 본 모델에서 사용한 방식으로, 가장 직관적이고 가독성이 좋다.

```

for (i in 1:N) {
  real mu_group = (structural[i] == 1)
    ? mu_sPT[vertebra_level_numeric[i], side_numeric[i]]

```

```

      : mu_nonsPT[vertebra_level_numeric[i], side_numeric[i]];
width[i] ~ normal(mu_group, sigma);
}

```

- * **Explicit Log-Probability (target +=) 방식:** 위의 코드는 내부적으로 전역 변수인 target에 로그 확률 밀도를 더하는 아래의 코드와 완벽히 동일하게 동작한다.

```

for (i in 1:N) {
  real mu_group = (structural[i] == 1)
    ? mu_sPT[vertebra_level_numeric[i], side_numeric[i]]
    : mu_nonsPT[vertebra_level_numeric[i], side_numeric[i]];
  target += normal_lpdf(width[i] | mu_group, sigma);
}

```

- * **Vectorization (벡터화) 방식:** 루프를 사용해 하나씩 더하는 것보다, 데이터 전체를 한꺼번에 계산하는 벡터화 방식이 연산 효율성 면에서 더 권장된다.

```
target += normal_lpdf(width | mu_mapped, sigma);
```

단, 이 방식을 사용하려면 transformed parameters 블록 등에서 각 데이터 i 에 대응하는 평균값, 즉 $\text{structural}[i]$ 의 값에 따라 mu_sPT 또는 mu_nonsPT 에서 선택된 값을 미리 나열한 mu_mapped 벡터를 생성하는 과정이 선행되어야 한다.

- ▷ (참고) 현재의 stan 파일은 모든 환자가 동일한 평균을 가진다고 가정하는 pooling 모델에 가깝다. 이와 대비되는 모델이 환자별 개인차를 반영할 수 있는 계층적 모델(hierarchical model)이다.

계층적 모델은 따로 다룰 것이다. (ch24. 계층 모델을 참고하기 바란다.)

0.2.0.1 Generated quantities 블록(GQB)

- GQB는 Stan file의 블록 중 하나이지만 워낙 중요하여 따로 subsection으로 분리하여 설명한다.
- GQB에 대해서는 다음을 참조하기 바란다.
 - ▷ GQB in 정규 분포: 내 책 227페이지
 - ▷ GQB in 포아송 분포: 내 책 251페이지
- 본 모델의 GQB는 다섯 가지 계산을 수행한다.
 - a. 사후예측분포(PPD) 생성: y_{rep}
 - b. 모델 적합성 평가: Bayesian p 값
 - c. 그룹별 2mm 미만 확률 계산: prob_narrow_sPT , $\text{prob_narrow_nonsPT}$
 - d. 그룹 간 척추경 폭 차이 계산: mu_diff

e. LOO-CV용 로그 가능도 계산: `log_lik`

- a. 사후예측분포(PPD) 생성: `y_rep`

- ▷ GQB의 가장 중요한 기능이다. 샘플링 단계에서 생성된 $\mu^{\text{sPT}}, \mu^{\text{nonsPT}}, \sigma$ 가 한 세트를 이루는 4000개의 모수 세트가 GQB로 전달된다.
- ▷ 모수 4000세트에 대해서 각각 2020개의 예측치를 만들면, 그 결과물(`y_rep`)은 행 4000개, 열 2020개의 거대한 행렬이 된다.
- ▷ `y_rep`의 구조를 보면,
 - * 행(Rows, 4,000개): MCMC 알고리즘이 생성한 4,000개의 사후 샘플(Iteration)이다.
 - * 열(Colums, 2,020개): 각 샘플마다 생성된 2,020개의 예측치(`y_rep`)이다.
- ▷ 2020은 사후분포의 개수가 아니라 데이터의 개수이다. 이 숫자가 어떻게 나왔는지에 대해서는 앞에서 기술한 바 있다. 그리고 data.R의 마지막 부분 “stan_data 구조 확인”에서 다시 한번 확인할 수 있다.
- ▷ 지금까지 얘기한 내용을 실행하는 코드는 GQB에 있는 다음 코드이다.

```
// 1. 사후예측 (모델 검증용)
array[N] real y_rep;
for (n in 1:N) {
  real mu_group = (structural[n] == 1)
    ? mu_sPT[vertebra_level_numeric[n], side_numeric[n]]
    : mu_nonsPT[vertebra_level_numeric[n], side_numeric[n]];
  y_rep[n] = normal_rng(mu_group, sigma);
}
```

if-else 문을 사용하는 대신 (가능도 부분에서 설명한) 삼항 연산자를 사용한 것을 알 수 있다.

- ▷ model 블록의 가능도 코드와 구조가 동일하되, `width[n] ~ normal(...)`(관측치의 확률 평가) 대신 `normal_rng(...)`(난수 생성)을 사용한다는 점이 다르다.
- ▷ 삼항 연산자로 `structural[n]`의 값에 따라 μ^{sPT} 또는 μ^{nonsPT} 중 적절한 평균을 선택한 뒤 예측치를 생성한다.
- ▷ 아래는 PT curve를 sPT, nonsPT로 구분하지 않은 잘못된 코드이다.

```
array[N] real y_rep;
for (n in 1:N) {
  y_rep[n] = normal_rng(mu[vertebra_level_numeric[n],
    side_numeric[n]], sigma);
}
```

- b. 모델 적합성 평가(Bayesian p 값)

- ▷ “ y_{rep} 통계량이 관측된 데이터(실제 width)에 비하여 얼마나 다른지” 확인하는 변수를 만들어 베이지안 p 값을 구한다. 빈도주의 p 값이 유의성 검증을 하는데 반하여 베이지안 p 값은 유의성 검증이 아니라 모델 적합성을 보는 지표이다.
- ▷ (참고) Bayesian p 값과 별개로 유의성 검증을 하는 지표를 여기서 볼 수 있다. 밑에서 구하는 밑에서 구하는 파생 변수 `prob_narrow_sPT`, `prob_narrow_nonsPT`이다. 이 수치가 0.95 라면 “평균 척추경 폭이 2mm 이하일 확률이 95%라는 뜻이며, 이는 빈도주의의 $p < 0.05$ 와 유사한 임상적 의사 결정 근거가 된다.
- ▷ Bayesian p 값을 구하는 코드는 다음과 같다.

```
real mean_y_rep = mean(y_rep);
real mean_width = mean(to_vector(width));
int p_val      = (mean_y_rep > mean_width);
```

- * `width`는 관측 데이터이므로 샘플링과 무관하게 항상 2020개로 4000번의 샘플링 동안 변하지 않는 고정값이다.
- * `mean_width = mean(to_vector(width))`는 2020개 숫자의 평균이므로 숫자 1개이다. 데이터가 변하지 않으므로 샘플링과 관계없이 동일한 값이다.
- * `y_rep`는 4000번의 매 샘플링마다 2020개가 생성된다. `mean(y_rep)`는 2020개의 `y_rep`의 평균으로 총 4000개가 생성된다.
- * 4000개의 `mean(y_rep)`와 `mean_width`를 비교하여 `mean_y_rep > mean_width`의 비율을 p 값로 한다. p 값이 0.3 ~ 0.7이면 모델이 데이터를 잘 설명하는 것으로 판단한다.

• c. 그룹별 2mm 미만 확률: `prob_narrow_sPT`, `prob_narrow_nonsPT`

- ▷ 본 연구의 핵심 임상 질문인 “척추경 폭이 위험 수준 (2mm 미만)일 확률”을 sPT 그룹과 non-sPT 그룹 각각에 대해 계산한다. GQB에서 계산하는 파생변수이다.
- ▷ 이를 실행하는 코드는 다음과 같다.

```
matrix[6, 2] prob_narrow_sPT;
matrix[6, 2] prob_narrow_nonsPT;
for (k in 1:6) {
  for (s in 1:2) {
    prob_narrow_sPT[k, s] = (mu_sPT[k, s] < 2.0);
    prob_narrow_nonsPT[k, s] = (mu_nonsPT[k, s] < 2.0);
  }
}
```

- ▷ 예를 들어 T4 Left의 sPT 그룹 평균을 $\mu^{sPT}[4, 1]$ 이라고 표기할 수 있다. 이 값이 2.0mm 미만이면 1(TRUE), 이상이면 0(FALSE)을 반환한다.
- ▷ 행렬의 각 성분이 0 또는 1로 기록된 [6,2] 행렬이 4000개 쌓인 후, 어떤 성분에서 4000번 중 3800번이 1(TRUE)로 판정되었다면, 그 성분의 해당 척추경의 위험 확률(척추경의 폭이 2mm 미만일 확률)은 95%로 추정된다.

- ▷ 본 연구에서 문턱값을 2.0mm로 설정한 것은 필자가 2011년에 발표한 연구[Lee et al., 2011]에서 횡경(transverse diameter)이 2mm 미만인 경우를 ‘극소 척추경(Extremely small pedicle)’으로 정의하고 특수한 삽입 기법(medial margin targeting method)의 필요성을 역설했던 임상적 근거를 바탕으로 한다.

- d. 그룹 간 차이: `mu_diff`

- ▷ Gemini 버전에는 없던 새로운 파생변수로, 본 모델의 핵심 발견을 직접 정량화한다.
- ▷ 이를 실행하는 코드는 다음과 같다.

```
matrix[6, 2] mu_diff;
for (k in 1:6) {
  for (s in 1:2) {
    mu_diff[k, s] = mu_sPT[k, s] - mu_nonsPT[k, s];
  }
}
```

- ▷ $\text{mu_diff}[k, s] = \mu_{ks}^{\text{sPT}} - \mu_{ks}^{\text{nonsPT}}$ 이므로, 음수(-)이면 sPT 그룹의 척추경이 non-sPT 그룹보다 더 좁다는 것을 의미한다.
- ▷ 4000개의 사후 샘플 각각에서 이 차이가 계산되므로, `mu_diff`의 사후분포를 통해 두 그룹 간 차이의 불확실성까지 정량화할 수 있다.

- e. LOO-CV용 로그 가능도: `log_lik`

- ▷ 모델 비교에 사용되는 변수로, `loo` 패키지의 입력값으로 활용된다.
- ▷ 이를 실행하는 코드는 다음과 같다.

```
vector[N] log_lik;
for (n in 1:N) {
  real mu_group = (structural[n] == 1)
    ? mu_sPT[vertebra_level_numeric[n], side_numeric[n]]
    : mu_nonsPT[vertebra_level_numeric[n], side_numeric[n]];
  log_lik[n] = normal_lpdf(width[n] | mu_group, sigma);
}
```

- ▷ model 블록의 가능도 코드와 구조가 동일하되, 샘플링(~) 대신 `normal_lpdf`로 로그 확률 밀도값을 명시적으로 저장한다는 점이 다르다.

0.3 메인 코드

메인 파일명은 `main_analysis.R`로 [GitHub 저장소](#)의 `simple.ais` 폴더에 있다. 전체 코드는 너무 길어서 [GitHub 저장소](#)를 참고하기 바라며, 중요한 부분을 분리해서 다루기로 한다.

중요한 코드를 분석해보자.

- `source(here("R", "data.R"))`:

- ▷ `here()`는 () 안의 경로를 생성하는 함수이다. 이 경로는 프로젝트 루트의 경로이다. 따라서 `here("R", "data.R")`는 프로젝트 루트 기준으로 `R/data.R` 파일의 경로를 만드는 것이고,
- ▷ `source()`는 그 경로의 R 스크립트 파일을 불러와서 실행하라는 뜻이다.

- `stan_file <- here("stan", "simple_model.stan")`:

- ▷ 프로젝트 루트 기준으로 `stan/simple_model.stan` 파일의 경로 문자열을 생성하여,
- ▷ 그 경로를 `stan_file`이라는 변수에 저장하라는 뜻이다.
- ▷ 이후 코드에서 stan 모델 파일 경로를 반복해서 쓸 때 이 변수를 사용하기 위한 것이다.

- `if (!file.exists(stan_file)) {`
`stop(paste("Stan 파일을 찾을 수 없습니다.\n확인 경로:", stan_file))`
`}`

- ▷ `file.exists(stan_file)`: `stan_file` 경로에 파일이 실제로 존재하는지 확인하여 (TRUE/FALSE 반환)
- ▷ 존재하지 않으면 TRUE를 반환한다. 이 경우 `stop(...)`은 실행을 강제 중단시키고,
- ▷ `paste(...)`: 에러 메시지를 사용자에게 알려준다.
- ▷ 즉, Stan 파일이 없을 때 명확한 안내 메시지와 함께 조기에 실행을 멈추는 방어 코드이다.

- 다음은 `stan()` 함수로 적합을 하여 fit 객체를 생성하는 명령어이다.

```
fit <- stan(
  file   = stan_file,
  data   = stan_data,
  iter   = 2000,
  chains = 4,
  seed   = 42
)
```

적합을 할 때 여기서는 `rstan` 패키지의 `stan()` 함수를 사용했지만, `brms` 패키지를 사용하는 경우 `brm()` 함수를 사용하게 된다.

- 적합하는데 시간이 걸리므로 생성된 fit 객체를 다음과 같이 `saveRDS()` 함수를 이용하여 .rds 파일로 저장해 놓으면 나중에 fit 객체가 필요한 경우 불러서 즉시 사용할 수 있다.

- ▷ .rds 파일로 저장: fit 객체→fit_simple.rds 파일
`saveRDS(fit, here("output", "fit_simple.rds"))`: 이 코드는 “적합으로 생성된 fit 객체를 output 폴더 내의 fit_simple.rds 파일로 저장한다”는 뜻이다.

▷ 나중에 (재사용 시) 불러오기: fit_simple.rds 파일→fit 객체
`fit <- readRDS(here("output", "fit_simple.rds"))`

- 바로 위의 코드

`dir.create(here("output"), showWarnings = FALSE, recursive = TRUE)`는

- ▷ `here("output")`는 프로젝트 루트 기준으로 output/ 폴더 경로를 생성한다는 뜻이고,
- ▷ `dir.create(here(...))`는 해당 경로에 폴더를 생성한다는 뜻이다.
- ▷ 따라서 fit_simple.rds 파일을 담을 output 폴더를 만드는 코드다.

- `stan()` 함수로 적합(fitting)을 하고 나면, **사후분포**의 샘플 정보가 담긴 fit 객체가 생성된다.

- ▷ 사후분포는 모델이 데이터를 학습한 후 업데이트한 모수(파라미터, parameter) 값들이다.
- ▷ 이 모델의 모수는 sPT 그룹과 non-sPT 그룹 각각에 대해 T1~T6의 Concave/Convex 척추경의 평균 넓이인 μ (각 그룹별 12개, 총 24개), 두 그룹 간 차이인 μ_{diff} (12개), 그리고 전체적인 오차 범위(데이터의 변동성)를 나타내는 표준편차 σ 이다.
따라서 사후분포의 모수의 개수는 μ_{sPT} 12개 + μ_{nonsPT} 12개 + σ 1개 등 총 25 개이다. (이를 모수 세트라고 하자.)
- ▷ Stan은 모수 세트의 평균값을 내놓는 것이 아니라 4000개의 샘플(모수 세트)을 내놓는다. (iteration=2000, chains=4, warm-up=1000)
- ▷ fit 객체에는 25개로 이루어진 모수 세트 외에도 generated quantities 블록(GQB)에서 정의된 변수들이 함께 저장된다. 여기에는 PPC(Posterior Predictive Check) 용도의 y_{rep} (N개), 두 그룹 간 평균 차이인 μ_{diff} (12개), 각 그룹의 2mm 미만 확률인 prob_narrow_sPT 와 $\text{prob_narrow_nonsPT}$ (각 12개), 그리고 모델 비교에 사용되는 $\log_{\text{lik}}$ (N개)가 포함된다. 이들은 모수 추정이 아니라 모델 검증 및 사후 분석 목적으로 생성된 값들이다.

- 사후분포를 확인하는 명령어들로 `print()`, `summary()`, `plot()`, `bayesplot()`, `mcmc_trace()` 등이 있다. 이 함수들의 괄호 안에 `fit`을 넣거나, 특정한 모수만을 볼 때는 '`fit, pars = c("mu", "sigma")`'를 괄호 안에 넣는다.

- `print(fit)`과 `summary(fit)`이 가장 많이 쓰인다. 둘 다 parameters 블록, GQB(generated quantities bloc)에서 정의된 변수들을 보여준다. (transformed parameters 블록이 있는 경우 이 블록에서 정의한 변수도 포함한다.)

- `print(fit)`과 `summary(fit)`의 목적은 다르다.

`print(fit)`은 분석결과를 일목요연하게 콘솔창에 정리해서 보여주는 것이 주목적이다.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu_sPT[1,1]	3.66	0.00	0.10	3.47	3.59	3.66	3.72	3.85	9926	1
mu_sPT[1,2]	3.63	0.00	0.10	3.44	3.56	3.63	3.70	3.83	8990	1
mu_sPT[2,1]	3.12	0.00	0.10	2.93	3.06	3.12	3.19	3.31	8635	1
mu_sPT[2,2]	2.36	0.00	0.10	2.17	2.29	2.36	2.43	2.56	9063	1
mu_sPT[3,1]	2.67	0.00	0.10	2.47	2.60	2.67	2.73	2.86	8751	1
mu_sPT[3,2]	1.32	0.00	0.13	1.08	1.23	1.32	1.40	1.56	9212	1
mu_sPT[4,1]	2.49	0.00	0.10	2.30	2.42	2.49	2.56	2.68	9963	1
mu_sPT[4,2]	1.12	0.00	0.15	0.84	1.02	1.12	1.22	1.41	8921	1
mu_sPT[5,1]	2.29	0.00	0.10	2.10	2.22	2.29	2.36	2.48	10750	1
mu_sPT[5,2]	1.23	0.00	0.12	1.00	1.15	1.23	1.31	1.46	9622	1
mu_sPT[6,1]	2.12	0.00	0.10	1.92	2.05	2.12	2.18	2.32	9548	1
mu_sPT[6,2]	1.80	0.00	0.10	1.61	1.74	1.80	1.87	1.99	10013	1
mu_nonsPT[1,1]	3.76	0.00	0.08	3.62	3.71	3.76	3.82	3.91	9570	1
mu_nonsPT[1,2]	3.74	0.00	0.08	3.60	3.69	3.74	3.79	3.89	9055	1
mu_nonsPT[2,1]	3.05	0.00	0.08	2.90	3.00	3.06	3.11	3.21	9891	1
mu_nonsPT[2,2]	2.50	0.00	0.08	2.35	2.45	2.50	2.56	2.66	9739	1
mu_nonsPT[3,1]	2.53	0.00	0.08	2.38	2.48	2.53	2.58	2.68	7496	1
mu_nonsPT[3,2]	1.40	0.00	0.09	1.23	1.34	1.40	1.46	1.56	10318	1
mu_nonsPT[4,1]	2.15	0.00	0.08	1.99	2.09	2.14	2.20	2.30	9411	1
mu_nonsPT[4,2]	1.04	0.00	0.08	0.87	0.98	1.04	1.09	1.21	9929	1
mu_nonsPT[5,1]	2.12	0.00	0.08	1.96	2.06	2.12	2.17	2.28	10781	1
mu_nonsPT[5,2]	1.61	0.00	0.08	1.44	1.55	1.61	1.66	1.76	8464	1
mu_nonsPT[6,1]	2.02	0.00	0.08	1.88	1.97	2.02	2.07	2.17	7588	1
mu_nonsPT[6,2]	1.87	0.00	0.08	1.72	1.81	1.87	1.92	2.01	8008	1
sigma	0.81	0.00	0.01	0.79	0.80	0.81	0.82	0.84	9131	1
y_rep[1]	3.77	0.01	0.82	2.17	3.22	3.76	4.33	5.38	4066	1
y_rep[2]	3.05	0.01	0.82	1.49	2.49	3.06	3.59	4.64	3935	1
y_rep[3]	2.54	0.01	0.82	0.94	2.00	2.53	3.10	4.13	4033	1
y_rep[4]	2.17	0.01	0.82	0.52	1.60	2.17	2.72	3.76	4021	1

- `summary()`는 결과를 ‘가공 가능한 데이터(List 형태)’로 반환하는 데 목적이 있다.

단순히 화면에 출력하는 것을 넘어, 평균, 표준편차, 신뢰구간 등의 수치를 행렬(Matrix) 형태로 변수에 저장할 수 있게 해준다.

```
> summary_fit
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu_sPT[1,1]	3.65516391	0.0009805049	0.09768746	3.465101e+00	3.58732139	3.65501829	3.72154648	3.85055057	9926.086317	1.0000521
mu_sPT[1,2]	3.63452946	0.0010578872	0.10030340	3.443458e+00	3.56398727	3.63419065	3.70350020	3.83129070	8989.852608	0.9995499
mu_sPT[2,1]	3.12210694	0.0010584086	0.09835133	2.930531e+00	3.05651394	3.12289035	3.18780229	3.30737673	8634.829986	0.9992229
mu_sPT[2,2]	2.35887600	0.0010537744	0.10032067	2.166901e+00	2.28830723	2.35856529	2.42903047	2.55889728	9063.283857	0.9991776
mu_sPT[3,1]	2.66814279	0.0010385066	0.09714826	2.472006e+00	2.60356142	2.66861508	2.73284151	2.86057039	8750.876303	1.0000822
mu_sPT[3,2]	1.31605986	0.0013068847	0.12543077	1.075895e+00	1.22881379	1.31773800	1.40268593	1.55965121	9211.569814	1.0000822
mu_sPT[4,1]	2.48669031	0.0009910545	0.09892185	2.297080e+00	2.41683201	2.48790498	2.55517335	2.67640716	9962.982919	0.9996106
mu_sPT[4,2]	1.12148523	0.0015502423	0.14642030	8.379116e-01	1.02111554	1.12300847	1.22004462	1.40732452	8920.792517	0.9994763
mu_sPT[5,1]	2.29131854	0.0009509883	0.09860118	2.100648e+00	2.22398007	2.29100642	2.35739273	2.48423078	10750.134521	0.9990875
mu_sPT[5,2]	1.23131854	0.0011932292	0.11704407	9.982230e-01	1.15419067	1.23406918	1.31157312	1.45752542	9621.682873	0.9997024
mu_sPT[6,1]	2.11807686	0.0010320303	0.10084139	1.916637e+00	2.05260079	2.11864807	2.18453277	2.31578945	9547.569085	0.9996563
mu_sPT[6,2]	1.80238029	0.0009867645	0.09874095	1.608625e+00	1.73706749	1.80330291	1.86773504	1.99347473	10013.078146	0.9994251
mu_nonsPT[1,1]	3.76306785	0.0007852224	0.07681346	3.615862e+00	3.71138382	3.76191949	3.81556747	3.91423611	9569.500884	0.9990700
mu_nonsPT[1,2]	3.74202054	0.0007936700	0.07552433	3.595904e+00	3.69039710	3.74141558	3.79211861	3.89999729	9055.113221	0.9992414
mu_nonsPT[2,1]	3.05495539	0.0008120221	0.08075802	2.895133e+00	3.00032291	3.05538981	3.11015029	3.21434602	9890.893830	0.9998633
mu_nonsPT[2,2]	2.50436699	0.0008017981	0.07912593	2.347382e+00	2.45153233	2.50482830	2.55960440	2.65714125	9738.850076	0.9991913
mu_nonsPT[3,1]	2.53116343	0.0008710296	0.07541357	2.382993e+00	2.48061503	2.52963613	2.58349827	2.68084578	7496.061438	0.9997098
mu_nonsPT[3,2]	1.39688249	0.0008387300	0.08519820	1.234179e+00	1.33751525	1.39733657	1.45505123	1.56327549	10318.498250	0.9996320
mu_nonsPT[4,1]	2.14611351	0.0007959327	0.07721293	1.992652e+00	2.09471788	2.14495718	2.19842571	2.29748255	9410.817714	0.9994477
mu_nonsPT[4,2]	1.03561869	0.0008355494	0.08325920	8.704148e-01	0.97950148	1.03598087	1.09068668	1.20549388	9929.334577	0.9993618
mu_nonsPT[5,1]	2.11699322	0.0007858459	0.08159530	1.958508e+00	2.06124383	2.11670884	2.17214574	2.27789661	10780.912731	0.9992000
mu_nonsPT[5,2]	1.60648451	0.0008777721	0.08075339	1.444897e+00	1.55354665	1.60725366	1.65989818	1.76224909	8463.656374	0.9993344
mu_nonsPT[6,1]	2.02459480	0.0008645269	0.07530997	1.877292e+00	1.97437860	2.02445601	2.07379957	2.17309515	7588.361094	0.9992262
mu_nonsPT[6,2]	1.86646600	0.0008471684	0.07581292	1.722160e+00	1.81456437	1.86582145	1.91916406	2.01090069	8008.420951	0.9998760
sigma	0.81351109	0.0001348600	0.01288705	7.886998e-01	0.80481795	0.81344993	0.82204522	0.83885525	9131.474237	0.9997806
y_rep[1]	3.76792249	0.0128927308	0.82214183	1.727289e+00	3.21549948	3.75808594	4.33456629	5.38311136	4066.393691	1.0002931
y_rep[2]	3.05416928	0.0130054792	0.81583093	1.485709e+00	2.49382575	3.06147148	3.59418771	4.64078074	3935.026054	0.9995291
y_rep[3]	2.53573599	0.0128668066	0.81711696	0.942719e+00	1.99935612	2.53214988	3.09696978	4.12766239	4032.987706	0.9995954
y_rep[4]	2.16625266	0.0129568441	0.82159132	0.523439e+00	1.59523152	2.17153938	2.72262228	3.75868516	4020.806855	1.0003733

- `summary_fit <- summary(fit)$summary:` `summary(fit)`는 여러 통계 정보를 담고 있는 리스트(List) 형태로, `summary`, `c.summary` 행렬로 구성되어있다. (리스트는 일종의 서랍장이고

그 속에 서로 다른 형태의 데이터(표, 숫자, 문자 등)를 담아놓은 구조라고 생각하면 된다.)

이 중 실제 분석 수치들이 표 형태로 정리된 `summary` 행렬만을 추출하여 `summary_fit`에 저장 하라는 뜻이다.

- # `mu_sPT`만 골라내기

```
mu_sPT_summary <- summary_fit[grepl("mu_sPT\\[", rownames(summary_fit)), ]
sorted_sPT <- mu_sPT_summary[order(mu_sPT_summary[, "mean"]),
                              c("mean", "sd", "2.5%", "97.5%", "Rhat")]
```

```
cat("=== sPT 그룹 mu 결과 (오름차순) ===\n")
```

```
print(round(sorted_sPT, 3))
```

거대한 도서관(`summary_fit`)에서 만약 `mu_sPT`에 관한 데이터만 보려면 다음 코드를 사용한다. 아주 유용한 코드이다.

- ▷ `grep("mu\\[", ...)`: `grep`은 수많은 텍스트 중에서 특정 글자 패턴을 찾는 탐정 역할을 하는데 여기서는 “ `mu[` ”까지만 정확히 찾으라고 명령하는 것이다. 여기서 `\\[`와 같이 대괄호 앞에 백슬래시 두 개를 붙이는 이유는 특정한 기능이 있는 대괄호가 아니라 진짜 대괄호를 찾으라고 알려주는 일종의 암호(Escape character)이다.
- ▷ 이렇게 찾은 `mu`의 위치 번호들을 가지고 원본 표에서 해당 줄(Row)들만 통째로 도려낸다.
- ▷ `rownames(summary_fit), ]`: 쉼표(comma) 뒤의 빈칸은 ‘해당 행의 모든 열(mean, sd, 신뢰구간 등)을 다 가져오라’는 뜻이다.
- ▷ `mu_sPT_summary`에서 `mu_sPT`의 오름차순(order)으로 보려면 다음 코드를 적용하면 된다.

```
sorted_sPT <- mu_sPT_summary[order(mu_sPT_summary[, "mean"]),
                              c("mean", "sd", "2.5%", "97.5%")]
```

- * `mu_sPT_summary[]`은 `mu_sPT_summary`에서 데이터프레임을 만드는데, 데이터프레임의 행은 `mu_sPT_summary`의 mean 열과 그 행 이름을 따서 mean 값의 오름 차순으로 정리하고

- * 데이터프레임의 열은 mean과 sd, 2.5%, 97.5%의 네 열로 만들라는 뜻이다.

- ▷ 이 코드의 결과 `sorted_sPT`는 다음과 같이 `mu_sPT`의 결과를 보여준다.

```

=== sPT 그룹 mu 결과 (오름차순) ===
> print(round(sorted_sPT, 3))
      mean    sd  2.5% 97.5% Rhat
mu_sPT[4,2] 1.121 0.146 0.838 1.407 0.999
mu_sPT[5,2] 1.231 0.117 0.998 1.458 1.000
mu_sPT[3,2] 1.316 0.125 1.076 1.560 1.000
mu_sPT[6,2] 1.802 0.099 1.609 1.993 0.999
mu_sPT[6,1] 2.118 0.101 1.917 2.316 1.000
mu_sPT[5,1] 2.291 0.099 2.101 2.484 0.999
mu_sPT[2,2] 2.359 0.100 2.167 2.556 0.999
mu_sPT[4,1] 2.487 0.099 2.297 2.676 1.000
mu_sPT[3,1] 2.668 0.097 2.472 2.861 1.000
mu_sPT[2,1] 3.122 0.098 2.931 3.307 0.999
mu_sPT[1,2] 3.635 0.100 3.443 3.831 1.000
mu_sPT[1,1] 3.655 0.098 3.465 3.851 1.000

```

- mu_nonsPT에 관한 데이터만 보려면 다음 코드를 사용한다.

```

# mu_nonsPT만 골라내기
mu_nsPT_summary <- summary_fit[grep("mu_nonsPT\\[", rownames(summary_fit)), ]
sorted_nsPT <- mu_nsPT_summary[order(mu_nsPT_summary[, "mean"]),
                                c("mean", "sd", "2.5%", "97.5%", "Rhat")]

```

```

cat("\n=== non-sPT 그룹 mu 결과 (오름차순) ===\n")
print(round(sorted_nsPT, 3))

```

- 만약 sPT와 non-sPT의 척추경의 폭의 차이를 비교하고 싶으면 코드의 따옴표 안을 다음과 같이 mu_diff로 바꾸면 된다.

```

# mu_diff 확인 (sPT - non-sPT, 음수 = sPT가 더 좁음)
mu_diff_summary <- summary_fit[grep("mu_diff\\[", rownames(summary_fit)), ]
cat("\n=== mu_diff (sPT - non-sPT) | 음수 = sPT가 더 좁음 ===\n")
print(round(mu_diff_summary[, c("mean", "sd", "2.5%", "97.5%")], 3))

```

결과는 다음과 같다.

```

=== mu_diff (sPT - non-sPT) - 음수 = sPT가 더 좁음 ===
> print(round(mu_diff_summary[, c("mean", "sd", "2.5%", "97.5%")], 3))
      mean    sd   2.5%  97.5%
mu_diff[1,1] -0.108 0.122 -0.346  0.133
mu_diff[1,2] -0.107 0.126 -0.355  0.142
mu_diff[2,1]  0.067 0.124 -0.174  0.309
mu_diff[2,2] -0.145 0.124 -0.384  0.095
mu_diff[3,1]  0.137 0.124 -0.106  0.376
mu_diff[3,2] -0.081 0.152 -0.376  0.213
mu_diff[4,1]  0.341 0.128  0.094  0.590
mu_diff[4,2]  0.086 0.170 -0.240  0.417
mu_diff[5,1]  0.174 0.130 -0.078  0.429
mu_diff[5,2] -0.375 0.142 -0.652 -0.098
mu_diff[6,1]  0.093 0.124 -0.152  0.337
mu_diff[6,2] -0.064 0.124 -0.308  0.175

```

- 각 레벨×측면에서 척추경의 폭이 2mm 미만일 사후 확률에 대해서 알아보자. GQB에서 정의했던 변수 prob_narrow_sPT, prob_narrow_nonsPT를 알아보는 것이다.

```

# --- 1. summary_fit에서 추출 ---
prob_narrow_sPT_summary <- summary_fit[grepl("prob_narrow_sPT\\\[",
      rownames(summary_fit)), ]
prob_narrow_nonsPT_summary <- summary_fit[grepl("prob_narrow_nonsPT\\\[",
      rownames(summary_fit)), ]

# --- 2. 행 이름을 "T1_Left" 형식으로 정리 ---
level_labels <- paste0("T", 1:6) # T1 ~ T6 (필요시 수정)
side_labels <- c("Left", "Right")

row_labels <- paste0(
  rep(level_labels, each = 2), "_",
  rep(side_labels, times = 6)
)

```

- ▷ 앞서와 같이 Grep(...)을 사용하여 prob_narrow_sPT, prob_narrow_nonsPT가 행 이름인 모든 행을 추출한다.
- ▷ mu_sPT[4, 1]을 T4_Left와 같이 바꾸기 위하여, level_labels를 T1~T6, side_labels를 Left, Right로 정한다. 여기서 가장 궁금한 점은 (이렇게 정한다고 해서) 어떻게 mu_sPT[4, 1]는 T4_Left와 매칭이 되고, mu_sPT[6, 2]는 T6_Right와 매칭이 되는가 하는 점이다. 저절로 될 리는 만무하고...
- ▷ 매칭되는 원리는 Stan이 matrix를 저장하는 다음의 내부 규칙을 따른다는 점이다.

Stan의 matrix 저장 규칙: 열(column) 우선

matrix[6, 2]는 내부적으로 이렇게 저장된다.

[1, 1] → [2, 1] → [3, 1] → [4, 1] → [5, 1] → [6, 1] ← 1열 다 끝난 후
 [1, 2] → [2, 2] → [3, 2] → [4, 2] → [5, 2] → [6, 2] ← 2열

▷ 결과는 다음과 같다.

```
=== prob_narrow_sPT (sPT 그룹: 폭 < 2mm 사후 확률) ===
> print(round(prob_narrow_sPT_summary[, c("mean", "sd", "2.5%", "97.5%")], 3))
```

	mean	sd	2.5%	97.5%
T1_Left	0.000	0.000	0	0
T1_Right	0.000	0.000	0	0
T2_Left	0.000	0.000	0	0
T2_Right	0.000	0.000	0	0
T3_Left	0.000	0.000	0	0
T3_Right	1.000	0.000	1	1
T4_Left	0.000	0.000	0	0
T4_Right	1.000	0.000	1	1
T5_Left	0.001	0.032	0	0
T5_Right	1.000	0.000	1	1
T6_Left	0.119	0.324	0	1
T6_Right	0.978	0.147	1	1

여기서 mean은 (GQB에서 설명한 바와 같이) 척추경의 폭(width)이 2mm 미만일 확률이다. T3, T4, T5의 우측 척추경은 (2018년 빈도주의 논문 결과와 마찬가지로) 모두 2mm 미만임을 알 수 있다.

- 이쯤 해서 fitting(적합)을 할 때 나타나야 할 변수가 어떤 종류, 몇 개가 있는지 정리해보자.

표 2: fit 결과에 포함되는 전체 변수 구성

블록 (Origin)	변수명	개수 계산	합계
Parameters	mu_sPT, mu_nonsPT, sigma	$(6 \times 2) + (6 \times 2) + 1$	25개
GQB	mu_diff	6×2	12개
GQB	prob_narrow_sPT, prob_narrow_nonsPT	$(6 \times 2) + (6 \times 2)$	24개
GQB	y_rep	N (데이터 개수)	2020개
GQB	log_lik	N (데이터 개수)	2020개
Internal	lp_--	로그 사후 확률 밀도	1개
총합			4102개

- ▷ `lp__` (Log Posterior): 코드에서 직접 선언하지 않았지만, Stan이 표본을 추출할 때마다 계산하는 내부 지표이다. 모델이 얼마나 데이터에 잘 수렴했는지 판단하는 척도가 된다.
- ▷ `prob_narrow_sPT[...]` 등의 NaN: 일부 값들이 NaN으로 나온다. 이는 4,000번의 샘플링 동안 해당 마디의 평균이 2mm 미만인 경우가 단 한 번도 없거나(확률 0), 혹은 반대로 모두 2mm 미만일 때(확률 1) 발생할 수 있는 현상이다.

0.3.1 시각화 결과

`posterior <- as.matrix(fit)`: 이 코드에 대해서 알아보자.

- `as.matrix()`, `print()`, `summary()`의 세 가지 명령어를 구분하는 것은 베이지안 데이터를 다루는 ‘눈’을 갖추는 과정과 같다. 쉽게 비유하자면 `as.matrix`는 요리하기 전의 ‘날것 재료(Raw material)’이고, `print/summary`는 요리가 끝난 ‘최종 보고서’라고 할 수 있다.

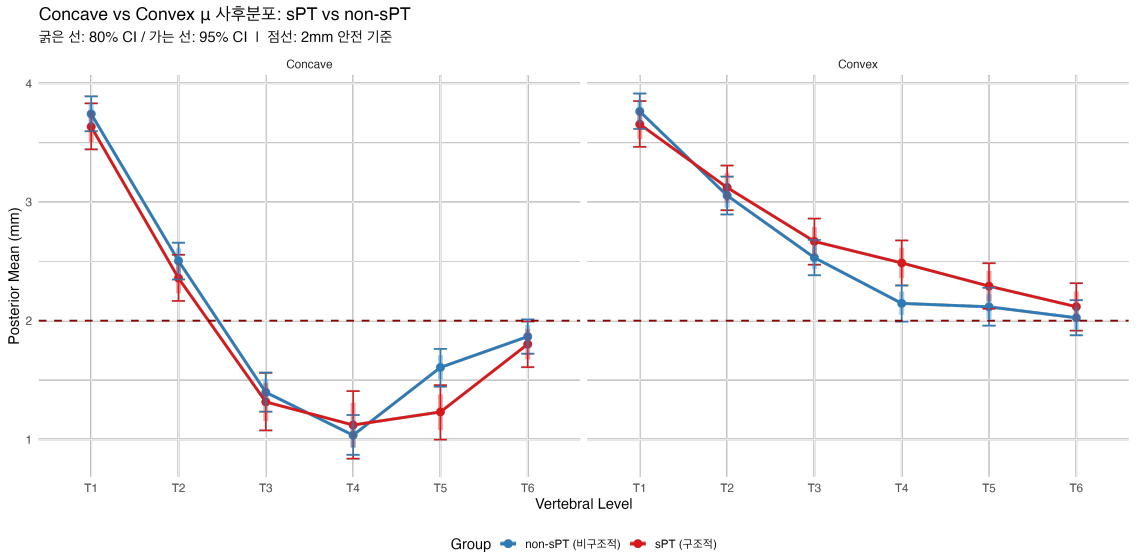
비교 항목	<code>as.matrix(fit)</code>	<code>summary(fit)</code>	<code>print(fit)</code>
성격	원시 데이터 (Raw Data)	통계 요약표 (Summary Stat)	콘솔 확인용 (Text Output)
데이터 행	4,000개 (시뮬레이션 횟수)	2,047개 (변수의 개수)	(<code>summary</code> 와 동일)
데이터 열	2,047개 (변수 이름)	10개 (mean, sd, quantiles...)	(<code>summary</code> 와 동일)
주 용도	직접 그래프를 그리거나 (히스토그램 등) 커스텀 계산을 할 때	표(Table)를 만들거나 분석 결과를 수치로 인용할 때	분석이 잘 끝났는지 (Rhat 등) 눈으로 빠르게 확인할 때

- `as.matrix(fit)` 명령어로 추출한 원시 데이터(Raw data)는 4,000개의 행(샘플)과 2,047개의 열(변수)을 가진 방대한 행렬이다. 4000은 `iterations=2000`, `chains=4`, `warmup=1000`에 따른 것이고, 2047은 앞서 언급한 전체 변수의 개수이다.
- 반면, 이를 요약한 `summary(fit)$summary`는 2,047개의 변수를 행(Row)으로 배치하고, 각 변수의 통계적 특성(평균, 표준편차, 신용구간 등)을 10개의 열(Column)로 정리하여 보여주는 ‘데이터 추출용’ 행렬이다.
- `print(fit)`은 `summary`와 동일하게 2,047개의 변수에 대한 통계치를 보여주지만, 결과값이 행렬 형태가 아닌 텍스트(Text) 형식으로 콘솔창에 출력된다. 이는 데이터 가공보다는 분석이 끝난 직후 Rhat이나 `n_eff` 등의 지표를 통해 모델의 수렴 여부를 빠르게 육안으로 확인하는 ‘모니터링용’으로 주로 사용된다.
- `posterior`의 차원인 `dim(posterior)`는 4000 2046이다. 왜 2047이 2046으로 바뀌었나?
Stan의 내부 진단 지표인 `lp__`가 행렬 추출 시 기본적으로 제외되기 때문이다. `lp__`까지 포함한 행렬을 만들려면 다음 코드를 이용한다.


```
# lp__를 포함하여 2,047개의 열을 모두 추출합니다.
posterior_all <- as.matrix(fit, pars =
  c("mu", "sigma", "mu_prob_narrow", "y_rep", "lp__"))
dim(posterior_all)
# [1] 4000 2047
```

이하 시각화 코드는 코드 분석 대신 결과만 보기로 한다.

A. sPT vs non-sPT Concave μ 비교 그래프



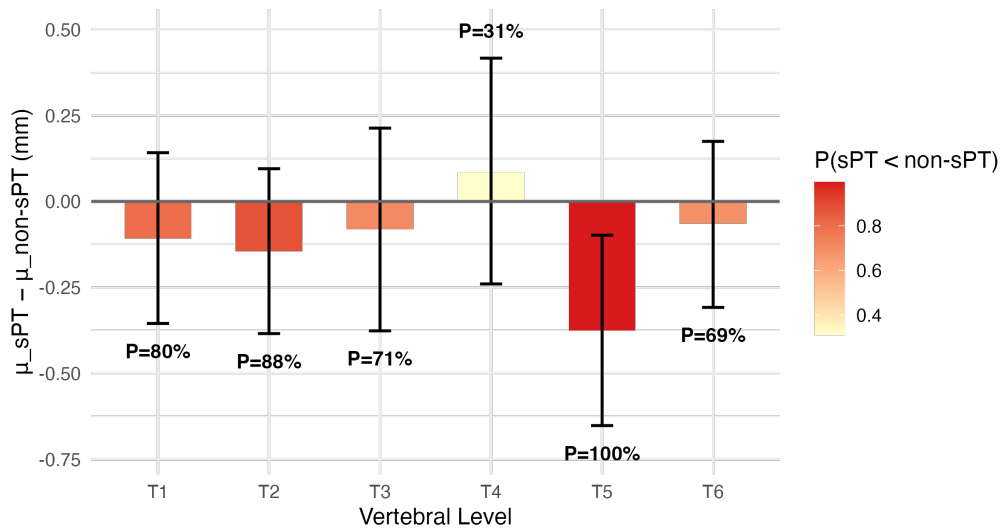
- Convex는 전반적으로 Concave보다 높은 위치를 유지하며, 대부분 구간에서 2mm 안전선 위에 있다. 따라서 concave T3, T4, T5에서 척추경이 좁아서 나사못을 못 박아도 더 중요한 convex side에서 나사못을 박을 수 있음을 보여준다.
- Concave side에서는 구조성 만곡의 척추경의 폭이 비구조성보다 전반적으로 작는데 반해서 convex side에서는 이런 차이를 보이지 않는다.

B. μ_{diff} 사후분포 (Concave side, sPT non-sPT)

- 아래 그림은 concave side(=right side in PT curve)에서 SPT와 non-sPT의 척추경의 넓이를 비교한 것이다. $sPT_{non} - sPT$ 를 구하여 음수로 나오는 경우 non-sPT의 척추경이 더 넓다는 의미이다.
- 두 그룹 간 차이는 T5에서 가장 크다. T4는 두 그룹 모두 좁고, T5는 특히 sPT에서만 좁은 패턴이다.

μ 차이(sPT - non-sPT) 사후분포 — Concave

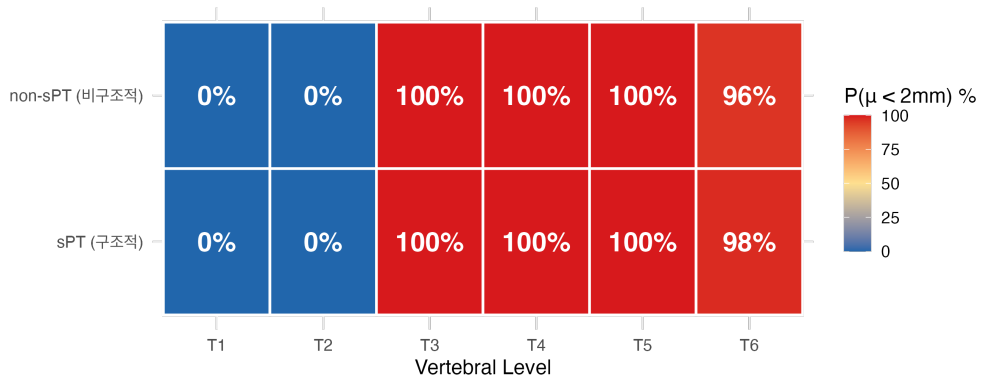
논문 $p=0.002(T3)$, $p=0.003(T4)$ 에 대응하는 베이지 결과
음수 = sPT가 더 좁음

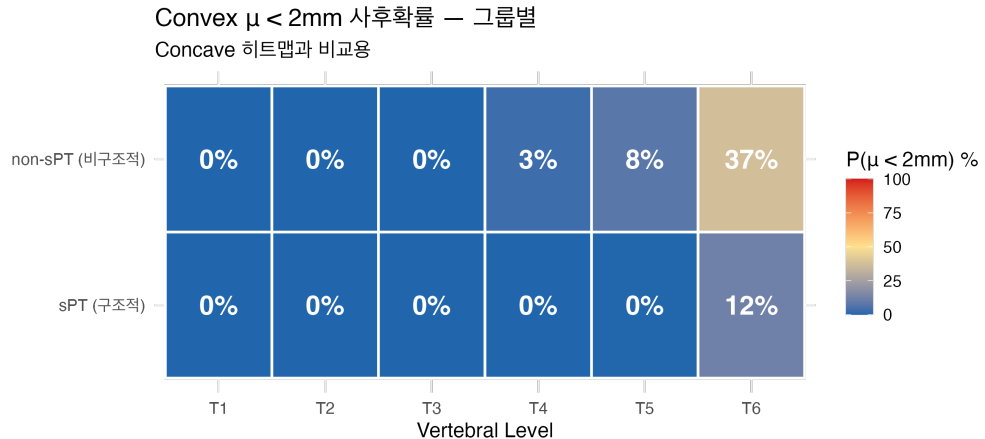


C. $P(\text{width} < 2\text{mm})$ 히트맵 — 그룹별

Concave $\mu < 2\text{mm}$ 사후확률 — 그룹별

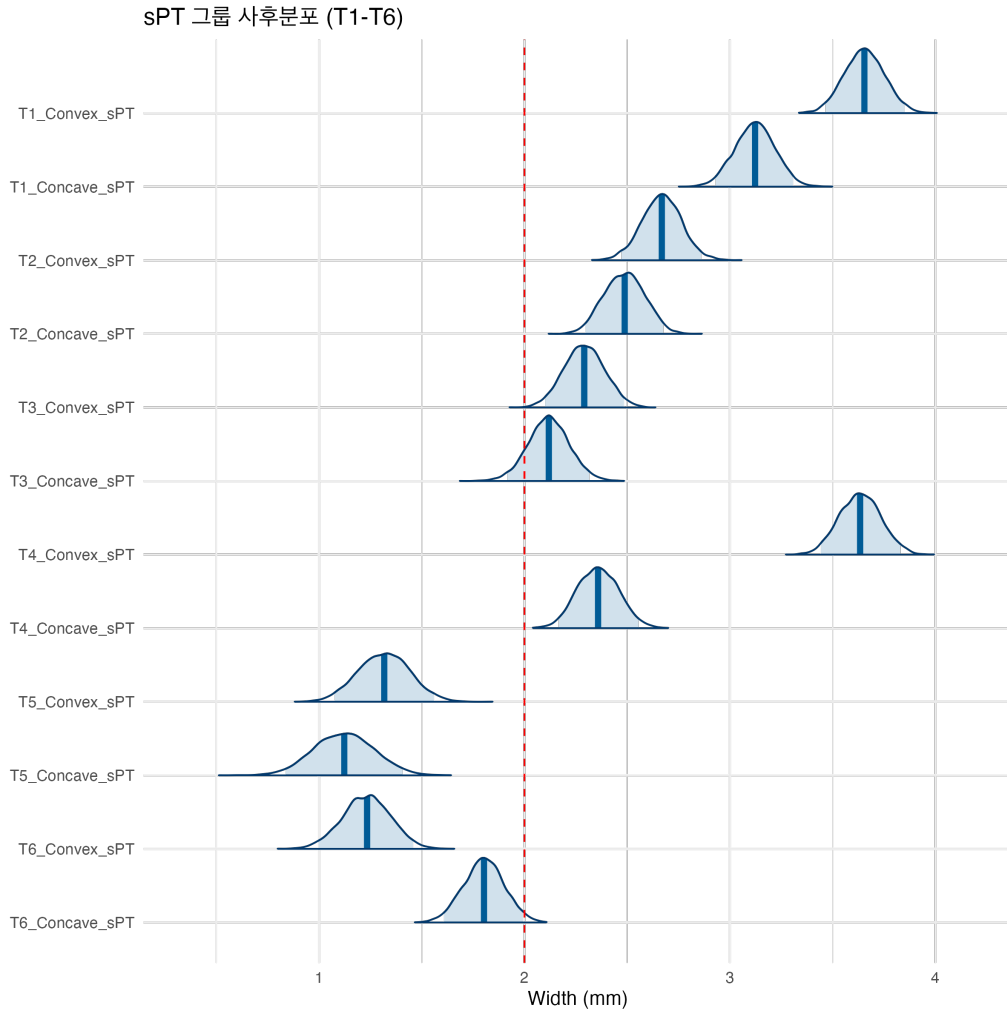
빨간색: 나사 삽입 위험. sPT의 T3/T4가 특히 위험





- 이 히트맵은 Concave와 Convex side에서 척추경 폭이 2mm 미만일 사후확률을 그룹별로 나타낸 것이다. Concave side에서는 sPT 및 non-sPT 두 그룹 모두 T3–T6에서 사후확률이 100%에 수렴하여, 이 구간에서의 나사 삽입이 구조적 진단 여부와 무관하게 위험함을 시사한다. 반면 Convex side에서는 T5까지 두 그룹 모두 10% 미만의 낮은 위험 확률을 보였으며, T6에서만 non-sPT 37%, sPT 12%로 상승하였다. 주목할 점은 T2에서 T3으로 넘어가는 구간에서 Concave 위험확률이 0%에서 100%로 급 전환된다는 것으로, 이는 수술 계획 시 나사 삽입 가능 레벨의 임상적 분기점으로 활용될 수 있다.
- 저자가 2021년 *Clinical Orthopaedics and Related Research*에 보고한 PT curve correction 술기에 따르면, Convex side는 main correction side로서 매 추체마다 나사못을 삽입하여 교정력을 확보해야 하며, Concave side는 supportive side로서 PT curve의 상단 및 하단 추체에만 나사못을 삽입하여도 충분한 고정을 얻을 수 있다.
- 히트맵의 소견은 위 술기의 해부학적 근거를 사후확률의 언어로 정확히 뒷받침한다. 다만 순서를 뒤집어 보면, 저자는 이 데이터 분석에 앞서 이미 임상 현장에서 PT curve correction을 시행하는 과정에서 경험적으로 이 술기의 필요성을 인식하고 있었다. 베イズ 분석은 그 경험적 판단이 해부학적 현실과 일치하고 있었음을 사후적으로 확인해 준 셈이다. 수술실에서 직관적으로 터득한 지식이 척추의 구조적 합리성과 맞닿아 있었다는 이 발견은, 임상 경험과 정량적 분석이 서로를 조명할 수 있음을 보여주는 드문 사례라 할 것이다.

D. 전체 사후분포 밀도 그래프



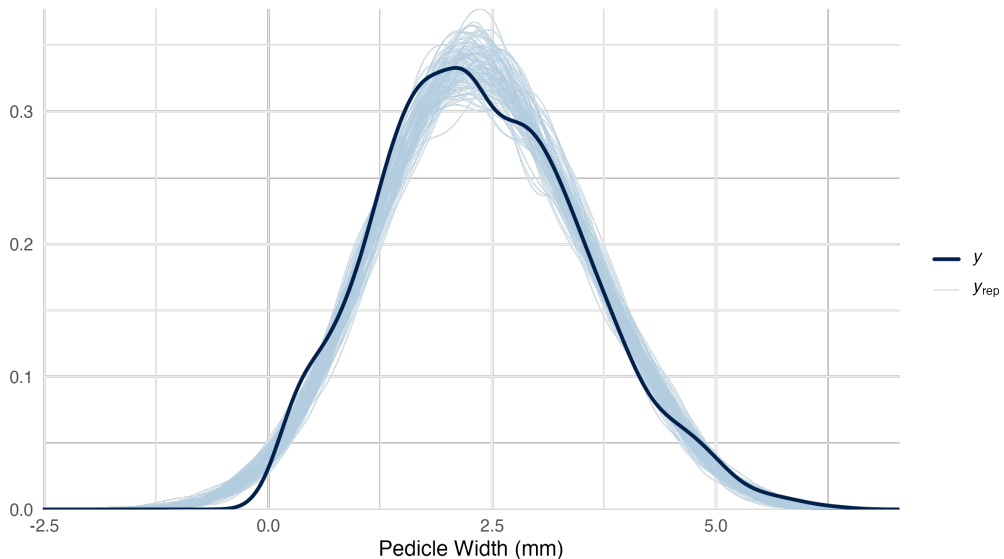
- 이 밀도 그래프는 sPT 그룹의 T1-T6 각 레벨에서 Convex와 Concave side의 척추경 폭 사후분포를 레벨별로 나란히 나타낸 것이다. 붉은 점선은 2mm 안전 기준선이며, 각 분포의 수직선은 사후 중앙값이다.
- **Convex vs. Concave 비교:** 모든 레벨에서 Convex 분포가 Concave 분포보다 일관되게 오른쪽(더 넓은 방향)에 위치한다. 이 패턴은 T3-T5에서 가장 두드러지며, T1과 T6에서는 두 분포 간 간격이 상대적으로 좁다.
- **2 mm 기준선 대비 위치:**
 - ▷ **T1-T2:** Convex와 Concave 모두 기준선 오른쪽에 위치 → 안전 구간
 - ▷ **T3:** Convex는 기준선 오른쪽에 위치하나, Concave는 기준선 쪽으로 이동 → 임상적 분기점
 - ▷ **T4-T5:** Convex는 기준선 근처에 위치하는 반면, Concave는 기준선보다 훨씬 왼쪽 (약 1mm대)에 위치 → Concave 고위험 구간

- ▷ **T6:** Convex는 기준선에 걸쳐 있고, Concave도 기준선에 근접하여 두 side 모두 주의 필요
- **분포의 폭(불확실성):** T1의 분포는 넓고 평탄한 반면, T3-T5의 Concave 분포는 좁고 뾰족하다. 이는 T1에서는 개인 간 변이가 크고, T3-T5 Concave에서는 척추경 폭이 좁은 범위로 수렴함을 의미한다. 즉 T3-T5 Concave의 위험은 일부 환자에 국한된 것이 아니라 sPT 환자 전반에 걸친 구조적 특성임을 시사한다.
- **임상적 함의:** T2에서 T3으로 넘어가는 구간이 Concave side의 임상적 분기점임이 사후분포로 명확히 확인된다. T3 이하 Concave 척추경에 나사를 삽입할 때는 술전 CT 측정과 영상 유도 시술이 필수적이며, 반면 Convex side는 T5까지 2mm 기준선을 상회하므로 상대적으로 안전한 나사 삽입이 가능함을 시사한다.

E. PPC (모델 검증)

PPC: Density Overlay

파란 선(예측)이 검은 선(실제)을 잘 감싸면 모델 적합



- 이 그림은 사후예측검증(Posterior Predictive Check, PPC) 결과로, 모델이 실제 데이터를 얼마나 잘 재현하는지를 시각적으로 확인한 것이다. 진한 검은 선(y)은 실제 관측 데이터의 밀도이며, 100개의 연한 파란 선(y_{rep})은 적합된 모델에서 사후예측분포를 통해 생성된 모의 데이터의 밀도이다.
- **전반적 적합도:** 100개의 y_{rep} 곡선이 실제 데이터 곡선(y)을 전반적으로 잘 감싸고 있어, 모델이 데이터의 전체적인 분포 형태를 적절히 포착하고 있음을 시사한다. 특히 분포의 중심부(약 2mm 근처)와 우측 꼬리 부분에서 예측 곡선들이 실제 곡선과 잘 일치한다.
- **좌측 꼬리의 경미한 불일치:** 0mm 근처에서 일부 y_{rep} 곡선이 음수 영역(-2.5mm)까지 뻗어 있는 반면, 실제 데이터(y)는 0mm에서 절단된다. 이는 모델이 정규분포를 가정하여 음수 척추경 폭을 허용하기 때문이며, 척추경 폭은 물리적으로 0mm 미만이 불가능하다는

점에서 모델의 구조적 한계이다. 다만 해당 영역의 확률질량이 매우 작아 주요 분석 결과에 미치는 영향은 미미하다.

- **결론:** PPC 결과는 본 모델이 척추경 폭 데이터를 합리적으로 설명하고 있음을 확인해 준다. 좌측 꼬리의 경미한 불일치는 향후 절단정규분포(truncated normal distribution)나 로그정규분포(log-normal distribution)를 적용하면 개선될 수 있으나, 현재 분석의 주요 결론인 그룹 간 척추경 폭 차이 및 위험 확률 추정에는 실질적인 영향을 미치지 않는다.

0.4 요약

빈도주의 분석과 베이지 분석의 비교: 무엇이 달라졌는가?: 2018년 *Spine*에 발표된 원 논문[Lee et al., 2018]은 182명의 환자 데이터를 바탕으로 빈도주의 통계(독립표본 t -검정, 다변량 선형회귀)를 적용하여 근위 흉추 척추경 폭을 분석하였다. 본 장에서는 동일한 데이터에 베이지 분석을 적용하여 얻은 결과를 원 논문과 비교하고, 두 접근법의 차이점과 베이지 변환을 통해 새롭게 얻은 통찰을 요약한다.

0.4.1 공통된 결론: 재현된 핵심 발견

베이지 분석은 원 논문의 핵심 결론을 모두 재현하였다.

- Concave side에서 T3, T4의 척추경 폭은 sPT 그룹이 non-sPT 그룹보다 좁다.
- T2 척추경 폭은 두 그룹 간에 차이가 없으며 2mm를 상회하여 안전하다.
- T1에서 T4 방향으로 폭이 좁아지다가 T4를 기점으로 다시 넓어지는 U자형 패턴이 확인된다.
- Convex side는 Concave side보다 전반적으로 척추경 폭이 넓다.

0.4.2 베이지 분석을 통해 새롭게 얻은 것

a. 점추정에서 분포로: 불확실성의 정량화

원 논문은 평균 $\pm$ 표준편차와 p 값으로 결과를 보고하였다. 예를 들어 sPT 그룹 T3 Concave의 척추경 폭은 0.76 ± 0.92 mm, p 값 = 0.002로 표현되었다. 베이지 분석은 이를 사후분포로 표현함으로써 “T3 Concave 척추경 폭의 95% 신용구간은 [하한, 상한]이며, 이 범위에 실제 모수가 포함될 확률이 95%이다”라고 직접적으로 해석할 수 있게 한다. 이는 빈도주의의 신뢰구간이 갖는 반직관적 해석 문제를 해소한다.

b. p 값에서 확률로: 임상적 위험의 직접 계산

원 논문은 그룹 간 차이의 유의성을 p 값(p 값 = 0.002, p 값 = 0.003)으로 표현하였다. 그러나 p 값은 “귀무가설이 참일 때 이 정도 이상의 차이가 관측될 확률”이며, “이 환자의 나사 삽입이 위험할 확률”을 직접 알려주지 않는다.

베이즈 분석은 이를 **사후확률**로 직접 계산한다. 히트맵이 보여주듯, T3-T6 Concave에서 척추경 폭이 2mm 미만일 사후확률은 두 그룹 모두 100%에 수렴한다. 이는 임상의가 수술 계획을 세울 때 “이 레벨에 나사를 박을 수 있는가”라는 질문에 확률의 언어로 직접 답해주는 것으로, *p*값이 제공할 수 없는 임상적 정보이다.

c. **T1-T4에서 T1-T6으로: 분석 범위의 확장**

원 논문은 T1-T4까지만 분석하였다. 이는 임상적으로 UIV(상위 고정 추체)가 T4까지 선택되는 경우가 대부분이기 때문이다. 베이즈 분석에서는 T5, T6까지 확장하여 분석하였으며, T5 Concave에서 sPT와 non-sPT의 차이가 가장 크게 나타나는 새로운 발견을 얻었다. 또한 T6 Convex에서 non-sPT(37%)가 sPT(12%)보다 오히려 위험 확률이 높은 역전 현상도 확인되었다.

d. **모델 검증의 명시화: PPC**

빈도주의 분석에서는 모델 가정의 적합성을 별도로 검증하지 않았다. 베이즈 분석은 사후예측검증(PPC)을 통해 모델이 실제 데이터를 얼마나 잘 재현하는지를 시각적으로 확인하였다. PPC 결과는 본 모델이 데이터를 합리적으로 설명함을 확인해 주었으며, 동시에 정규분포 가정으로 인한 좌측 꼬리의 경미한 불일치라는 모델의 한계도 투명하게 드러냈다.

e. **수술 술기와의 연결: 자연의 합리성 확인**

원 논문의 결론은 “T3, T4 Concave에서 나사 삽입이 어려울 수 있으므로 수술 계획 시 주의하라”는 것이었다. 베이즈 분석은 여기서 한 걸음 더 나아가, Concave side의 위험(T3-T6, 확률 100%)과 Convex side의 안전(T5까지 10% 미만)을 동시에 정량화함으로써, 저자가 2021년 *Clinical Orthopaedics and Related Research*에 보고한 PT curve correction 술기, 즉 “Convex side에서 교정하고 Concave side는 상하단만 고정한다”는 원칙의 해부학적 근거를 사후확률의 언어로 뒷받침하였다. 이는 임상 경험에서 귀납적으로 도달한 술기가 척추의 구조적 합리성과 일치하고 있었음을 정량적으로 확인한 것이다.

0.4.3 비교

표 3: 빈도주의 분석(2018)과 베이즈 분석의 비교

항목	빈도주의 (2018)	베이즈 (본 분석)
결과 표현	평균 ± SD, <i>p</i> 값	사후분포, 신용구간
불확실성	신뢰구간 (반직관적 해석)	신용구간 (직접적 확률 해석)
임상적 위험	간접적 (<i>p</i> 값으로 유추)	직접적 (위험 확률 %)
분석 범위	T1-T4	T1-T6
모델 검증	없음	PPC로 명시적 검증
핵심 결론	T3, T4 Concave 위험	T3-T6 Concave 위험 확률 정량화
추가 발견	-	T5에서 그룹 간 차이 최대, Convex side 안전성 정량화

참고 문헌

Choon Sung Lee, Soo-An Park, Chang Ju Hwang, Dong-Joon Kim, Won-Jae Lee, Yung-Tae Kim, Mi Young Lee, So Jung Yoon, and Dong-Ho Lee. A novel method of screw placement for extremely small thoracic pedicles in scoliosis. *Spine*, 36(16):E1112–E1116, 2011. doi: 10.1097/BRS.0b013e3181ffea2.

Choon Sung Lee, Jae Hwan Cho, Chang Ju Hwang, Dong-Ho Lee, Jae-Woo Park, and Kun-Bo Park. The importance of the pedicle diameters at the proximal thoracic vertebrae for the correction of proximal thoracic curve in asian patients with idiopathic scoliosis. *Spine*, 44(11):E671–E678, 2018. doi: 10.1097/BRS.0000000000002926.